Ethics - Software Development

Instructor: Varik Hoang

Platform: HuskyMap

Objectives

- Define clear quality standards aligned with organizational goals
- Secure leadership commitment and assign responsibilities
- Document processes and implement continuous improvement
- Provide training and support for employees
- Use data and metrics to monitor performance
- Focus on customer satisfaction and manage risks proactively

Organizations Behaving Badly

- Airlines depend on complex IT for flights, crews, check-in, baggage, and ticketing
- Systems are often **old and buggy**
- Failures can cause massive delays and revenue loss
- Industry consolidation
  - Four airlines handle 85% of U.S. flights
  - More complexity, higher risk
- Interconnected systems
  - One glitch can trigger **domino effects** across operations

Why It is Important

- **High-quality software**: fast, efficient, reliable, safe, and meets user needs
  - Critical in industries like air traffic control, healthcare, defense, and space
- **Software defects**: errors that can cause minor annoyances or major failures
- Notable examples of **software bugs**:
  - Nest thermostats failed during a cold spell → risk of frozen pipes
  - 2016 Blue Cross NC glitch → misbilling, coverage errors, unpaid providers
  - U.S. Customs shutdown → airport delays for travelers
  - fMRI software flaw → false brain activity results, impacting scientific studies

Software Quality & Management

- **Software quality**: degree to which a product meets user needs-fast, efficient, reliable, safe
- **Quality management**: defines, measures, refines quality in development and deliverables
- **Challenges**:
  - First releases rarely meet expectations; defects are normal
  - Developers may lack knowledge or time to design quality from the start
  - Systems must be resilient to human error and environmental factors
- **Defect statistics**:
  - Open source systems: **0**.**29** defects per **1**,**000** lines of code
  - Microsoft apps: ~**0**.**5** defects per **1**,**000** lines at release
- **Pressure to release**: time-to-market, revenue, competition, and shareholder expectations
  - Often reduce testing and quality assurance

Impacts, Ethics & Real-World Examples

- **Ethical dilemma**:
  - How much cost/effort should be spent ensuring software meets user expectations?
- **Low-quality apps**:
  - ~**22**% of **Android** apps fail in usability, security, reliability, compatibility, or efficiency
- **Market caution**: Many organizations avoid first releases; first major **Microsoft OS** versions (DOS, early Windows) were unstable
- **Unexpected failures**: Even reliable systems can fail when upgraded
  - Example: **British Airways**' 2016 check-in system update caused 5 major outages → thousands of flight delays/cancellations, £**92**.**9**B stock market loss

Why Software Quality Matters

- Business information systems rely on software to
  - Process transactions accurately and on time
  - Example: order-processing systems, airline ticketing, electronic funds transfer
- Software defects can **lose customers** and **reduce revenue**
- **Decision Support Systems** (**DSS**) help make forecasts, investments, and workers
  - Defects can have serious organizational consequences
- Industrial process-control software monitors operations, reduces errors, and improves quality
  - Defects can increase costs, waste, or cause unsafe conditions

Business & Ethical Implications

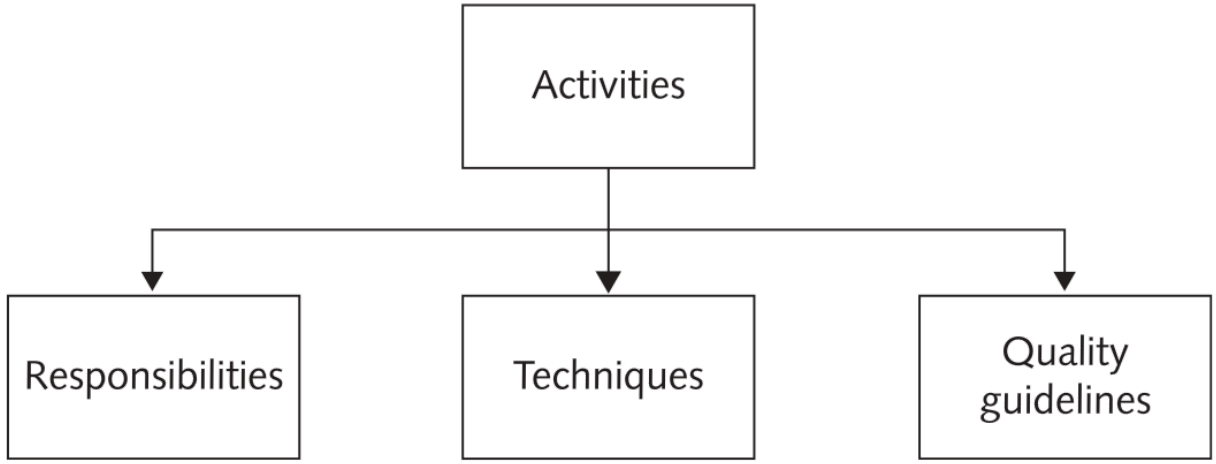
- Many products-from cars to medical devices-depend on software
  - Defects can range from minor inconveniences to serious safety risks
- Companies are increasingly in the **software business**
  - Making software quality critical to competitiveness and customer trust
- Executives face ethical decisions:
  - How much time and money should be spent on testing and quality assurance?
- Legal and product liability concerns increase as software controls critical systems

Software Product Liability

- **Product liability**:
  - Legal responsibility of manufacturers, sellers for injuries caused by defective products
- Applies to software that causes **physical harm**, **financial loss**, or **business disruption**
  - Examples: self-driving car malfunctions leading to accidents or property damage
- Claims are typically based on:
  - **Strict liability**
  - **Negligence**
  - **Breach of warranty**
  - **Misrepresentation**

Strategies for Developing Quality Software

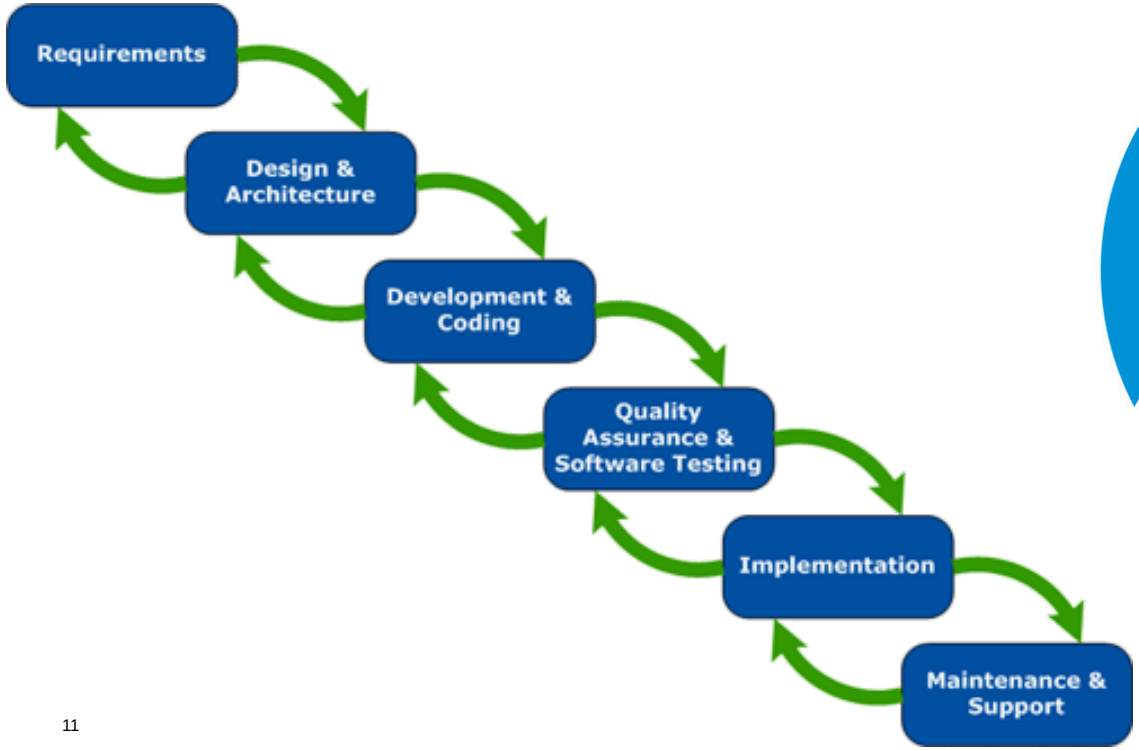
- **Software development methodologies**:
  - Standardized, proven processes guiding teams (analysts, programmers, testers, managers)
  - Define tasks, responsibilities, and quality management at each stage

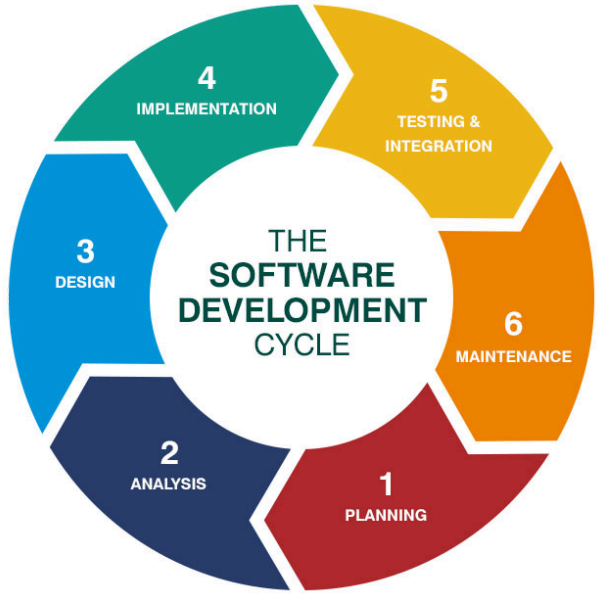


Software Development Methodologies

— **Waterfall Model**: sequential and multistage process

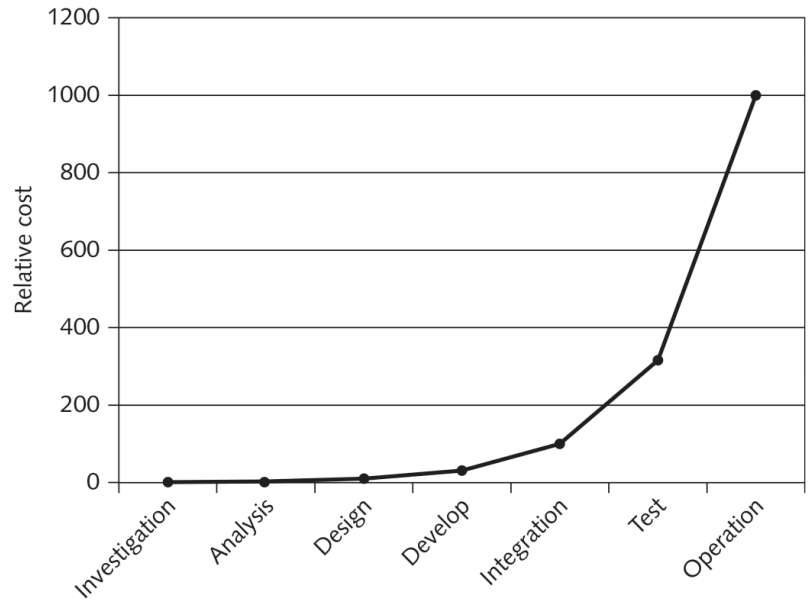
— **Agile Development**: maximizes flexibility and rapid delivery





Defect Detection and Legal Implications

- **Early defect detection**:
  - Fixing defects early can cost up to 100× less than after release
  - Later defects affect more people, require more communication, and increase costs
- **Legal implications**: effective methodologies reduces
  - Defects
  - Product liability risk



Quality Assurance (QA) and Quality Control (QC)

- **QA**: methods applied during development to ensure reliable, high-quality software
  - **Best practice**: QA at each stage of development
  - **Poor practice**: testing only before product release
  - *Focuses* on the product itself
- **QC**: process of inspecting, testing, and measuring products or services
  - Preventing defects during production
  - *Focuses* process of preventing defects

Software Testing

- **Dynamic Testing** (executing code with test data):
  - **Black-box testing**: tests input/output without knowing internal code
  - **White-box testing**: tests all logic paths with full knowledge of code
- **Static Testing** (code reviewed without execution):
  - **Review**: informal, walkthrough, peer review, inspection
  - **Static analysis**: detects unused code, security issues, and violations of standards
  - **Advantage**: defects detected early, reducing cost and effort
- **Other Testing** Types:
  - **Unit testing**: tests individual modules/subroutines
  - **Integration testing**: ensures modules work together correctly
  - **System testing**: tests the full system as a complete entity
  - **User acceptance testing**: trained users verify the system meets expectations

Capability Maturity Model Integration (**CMMI**)

- **Purpose**: Provides best practices to improve organizational processes and software quality
  - Developed by **industry**, **government**, and **Carnegie Mellon SEI**
- **CMMI-DEV**: specific for software development, covering **22 key process areas**
- **Five maturity levels**: identifying weaknesses and implementing improvement plans
  - Level 5 allows **statistical evaluation**, **continual improvement**, and **high-quality delivery**
- **Assessment**: uses trained, external assessors to evaluate practices objectively
- **Benchmarking**: **CMMI** level often required for federal software contracts
- **Business impact**:
  - High-maturity organizations deliver software faster, at lower cost, and with higher quality

Developing Safety-Critical Systems

- **Definition**: systems whose failure may cause **human injury or death**
- **Consequences of defects**: can be deadly, costly, and result in major recalls
- Examples:
  - **Toyota** recalls ~$**3**B for acceleration/braking defects
  - **Covidien** neonatal ventilators delivered incorrect air doses
  - **GM** airbags failed in **4**.**3M** vehicles due to software flaws
- **Development considerations**:
  - Standard methodologies are **not sufficient**
  - Safety-critical software requires **rigorous**, **time-consuming processes**
  - All stages: requirements, design, coding, testing, and change control, etc.
  - Developers must consider the **entire system**

Developing Safety-Critical Systems (cont.)

- **System safety engineer**:
  - Tracks hazards using a **hazard log** from start to finish
  - Conducts safety reviews and ensures safe system design, not just documentation
- **Ethical and practical dilemmas**:
  - Balancing **safety** vs. **cost**, **usability**, and **market competitiveness**
  - Determining *how much QA/testing is enough* when human lives are at risk.
- **Formal risk analysis**:
  - Assess what can go **wrong**, **likelihood**, and **consequences**
  - Implement measures to **prevent**, **mitigate**, **detect**, or **warn users** about risks

Risk Management in Software Development

- **Risk**: potential to gain or lose something of value
- **Key elements**:
  - **Risk event**: what might happen
  - **Probability**: likelihood of occurrence
  - **Impact**: effect on business outcome (positive or negative)
- **Metrics**:
  - **Annualized Rate of Occurrence** (**ARO**): probability event occurs in a year
  - **Single Loss Expectancy** (**SLE**): loss if the event occurs
  - **Annualized Loss Expectancy** (**ALE**): expected yearly loss:
    - ALE = ARO × SLE
- **Risk management**: process of identifying, monitoring, and limiting risks
- **Residual risk**: risk remaining after mitigation

Risk Management in Software Development (cont.)

- **Acceptance**
  - Accept risk when mitigation cost exceeds potential loss
  - Highly controversial in safety-critical systems
  - Raises ethical concerns
- **Avoidance**
  - Eliminate vulnerability causing risk
  - Most effective, but not always feasible
- **Mitigation**: reduce likelihood or impact of risk
  - Multiple independent versions run in parallel
  - Voting algorithm selects correct output
  - Common in aircraft and spacecraft systems
- **Redundancy**
  - Multiple interchangeable components
- **Transference**
  - Shift risk to another party
  - Insurance or outsourcing

Risk Management in Software Development (cont.)

- **Ethical Issues in Risk Decisions**
  - Product recalls often involve cost vs. safety trade-offs
  - Lawsuits vs. recall expenses sometimes weighed
  - Public trust may be damaged even without clear fault
- **Reliability vs. Safety**
  - **Reliability**: Probability system continues functioning
    - More components → lower overall reliability
  - **Safety**: Ability to operate without causing harm (system can be reliable but unsafe)
- **Human Factors & System Interface**
  - Humans are less predictable than hardware/software (poor interface design higher risk)
    - Prevent unsafe operator actions
    - Account for stress and crisis behavior

Quality Management Standards

- **International Organization for Standardization** (**ISO**)
  - Founded in **1947** and issued **9000** series in **1988**
- Purpose of ISO **9000** Series
  - Establish formal **quality management systems** (**QMS**)
  - Focus on meeting customer needs, expectations, and satisfaction
- ISO **9001** Family of Standards
  - Provides standardized requirements for a quality management system
  - Updated every five years to remain current
- Requirements for ISO **9001** Certification: document all processes in written procedures
- ISO **9001** and Software Development applied to: software purchase, development, etc.
- Provides structured quality control for software organizations

Failure Mode and Effects Analysis

Issue	Severity	Occurrence	Criticality	Detection	Risk priority
#1	3	4	12	9	108
#2	9	4	36	2	72
#3	4	5	20	4	80

- **Evaluates**:
  - System reliability
  - Effects of equipment/process failures
- **Goal**: Identify failures early, when correction is easier and less costly
- **Failure mode**: How a product/process could fail to meet customer requirements
- **Effect**: adverse consequence experienced by the customer
- **Complex systems**:
  - One cause → multiple effects
  - Multiple causes → one or more effects
- Typical FMEA may identify 50-200 potential failure modes

Checklist for Software Quality Improvement

- Organizations often prioritize issues with the **highest criticality**
  - Criticality = Severity × Occurrence
- Then consider **Risk Priority Number** (**RPN**)
  - RPN = Severity × Occurrence × Detection
- Important distinction:
  - An issue with lower RPN
  - But very high severity × occurrence
  - May still receive highest priority
- Management must use **judgment**, not just numbers

Questions & Answer


Thank You!